06.2: MongoDB Commands

UNIVERSITY OF MARYLAND

UMD DATA605 - Big Data Systems

06.2: MongoDB Commands

- **Instructor**: Dr. GP Saggese, gsaggese@umd.edu
- @References@:
  - All concepts in slides
  - MongoDB tutorial
  - Web
    - https://www.mongodb.com/
    - Official docs
    - pymongo
  - Seven Databases in Seven Weeks, 2e

1 / 11

2 / 11: CRUD Operations

CRUD Operations

- CRUD = Create, Read, Update, Delete
- **Create**

```
db.collection.insert(<document>)
db.collection.update(<query>, <update>, {upsert: true})
```

  - Upsert = update (if exists) or insert (if it doesn't)

- **Read**

```
db.collection.find(<query>, <projection>)
db.collection.findOne(<query>, <projection>)
```

- **Update**

```
db.collection.update(<query>, <update>, <options>)
```

- **Delete**

```
db.collection.remove(<query>, <justOne>)
```

- Details here

2 / 11

- **CRUD Operations**
  - **CRUD = Create, Read, Update, Delete**
    - CRUD operations are the basic functions of persistent storage. They are essential for interacting with databases, allowing you to manage data effectively.
- **Create**
  - `db.collection.insert(<document>)`
    `db.collection.update(<query>, <update>, {upsert: true})`
  - **Upsert**: This is a combination of update and insert. If the document you are trying to update does not exist, it will be inserted as a new document. This is useful for ensuring that data is present in the database without having to check if it exists first.
- **Read**
  - `db.collection.find(<query>, <projection>)`
    `db.collection.findOne(<query>, <projection>)`
  - Reading involves retrieving data from the database. The `find` method can return multiple documents that match a query, while `findOne` returns only the first matching document. *Projection* allows you to specify which fields to include or exclude in the result.
- **Update**
  - `db.collection.update(<query>, <update>, <options>)`
  - Updating is about modifying existing documents. You specify a query to find the documents you want to update and then define the changes. Options can include things like whether to update multiple documents or just one.

- **Delete**
  - `db.collection.remove(<query>, <justOne>)`
  - Deleting removes documents from the database. You can specify a query to determine which documents to delete. The `justOne` option allows you to delete only a single document even if multiple documents match the query.
- **Details here**
  - For more in-depth information, you can refer to the MongoDB documentation, which provides comprehensive details on how to perform these operations effectively.

3 / 11: Create Operations

Create Operations

- `db.collection` specifies the collection (like an SQL table) to store the document

  `db.collection.insert(<document>)`

  - Without `_id` field, MongoDB generates a unique key

    ```
    db.parts.insert({type: "screwdriver", quantity: 15})
    ```

  - Use `_id` field if it has a special meaning

    ```
    db.parts.insert({_id: 10, type: "hammer", quantity: 1})
    ```

- Update 1 or more records in a collection satisfying **query**

  `db.collection.update(<query>, <update>, {upsert: true})`

- Update an existing record or create a new record

  `db.collection.save(<document>)`

- A more modern OOP-like syntax than the COBOL / FORTRAN-inspired SQL

3 / 11

- **Create Operations**: In MongoDB, creating operations involve inserting documents into collections. A *collection* in MongoDB is similar to a table in SQL databases. This is where you store your data.
- **`db.collection` specifies the collection**: When you want to insert a document, you specify the collection where it should be stored. This is akin to choosing a table in SQL.

  ```
  db.collection.insert(<document>)
  ```

  - **Without `_id` field, MongoDB generates a unique key**: If you don't specify an `_id` field in your document, MongoDB will automatically create a unique identifier for it. This is useful for ensuring each document can be uniquely identified.

    ```
    db.parts.insert({type: "screwdriver", quantity: 15})
    ```

  - **Use `_id` field if it has a special meaning**: Sometimes, you might want to use a specific identifier for your document, especially if it has a special meaning or is used for

referencing.

```
db.parts.insert({_id: 10, type: "hammer", quantity: 1})
```

- **Update 1 or more records in a collection satisfying @query@**: You can update documents in a collection that match a specific query. The `update` function allows you to modify existing documents.

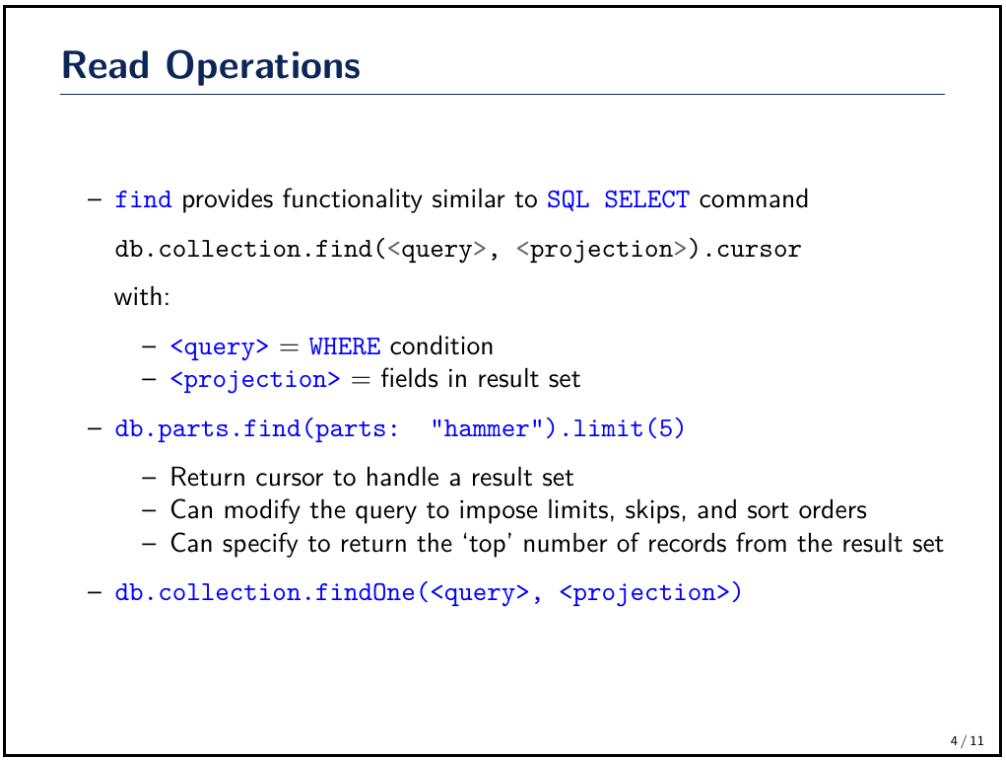
```
db.collection.update(<query>, <update>, {upsert: true})
```

- **Update an existing record or create a new record**: The `save` function is versatile. It updates an existing document if it exists, or creates a new one if it doesn't. This is useful for ensuring data consistency.

```
db.collection.save(<document>)
```

- **A more modern OOP-like syntax than the COBOL / FORTRAN-inspired SQL**: MongoDB's syntax is often considered more intuitive and object-oriented compared to traditional SQL, which has roots in older programming languages like COBOL and FORTRAN. This modern approach can make it easier to work with for developers familiar with object-oriented programming.

4 / 11: Read Operations



- **Read Operations**
  - The `find` function in databases like MongoDB is similar to the `SQL SELECT` command. This means it's used to retrieve data from a database.
    - The basic syntax is `db.collection.find(<query>, <projection>).cursor`. Here, `<query>` acts like the `WHERE` condition in SQL, which means it filters the data

to only include records that match certain criteria.
    - `<projection>` specifies which fields should be included in the results, similar to selecting specific columns in SQL.
- **Example**: `db.parts.find({parts: "hammer"}).limit(5)`
  - This command searches for documents in the `parts` collection where the `parts` field is "hammer".
  - It returns a *cursor*, which is a pointer to the result set. This allows you to iterate over the results.
  - You can modify the query to limit the number of results, skip certain records, or sort the results in a specific order. For instance, `limit(5)` restricts the output to the first five matching records.
- **Single Record Retrieval**: `db.collection.findOne(<query>, <projection>)`
  - This command is used when you want to retrieve only one document that matches the query criteria. It's useful when you expect or need only a single result.

5 / 11: More Query Examples

More Query Examples

- Mongo has a functional programming flavor
  - E.g., composing operators, like \textcolor{blue}{\texttt{$or}}

SQL

```
SELECT * FROM users WHERE age>33

SELECT * FROM users WHERE age!=33

SELECT * FROM users
  WHERE name LIKE "%Joe%"

SELECT * FROM users WHERE a=1 and b='q'

SELECT * FROM users WHERE a=1 or b=2

SELECT * FROM foo
  WHERE name='bob' and (a=1 or b=2 )

SELECT * FROM users
  WHERE age>33 AND age<=40
```

Mongo

```
db.users.find({age: {$gt: 33}})

db.users.find({age: {$ne: 33}})

db.users.find({name: /Joe/})

db.users.find({a: 1, b: 'q'})

db.users.find({$or: [{a: 1}, {b: 2}]})

db.foo.find({name: "bob",
  $or: [{a: 1}, {b: 2}]})

db.users.find({'age':
  {$gt: 33, $lte: 40}})
```

5 / 11

- **More Query Examples**
  This slide is about comparing how queries are written in SQL and MongoDB. It shows that MongoDB has a *functional programming flavor*, which means it uses a style where you can combine different operations together, much like functions in programming. An example of this is using operators like `$or` to combine conditions.
- **SQL Queries**
  - The SQL examples show how to select data from a table called `users` based on different conditions.

  - For instance, `SELECT * FROM users WHERE age>33` retrieves all users older than 33.
  - The `LIKE` operator is used for pattern matching, such as finding names containing "Joe".
  - Logical operators like `AND` and `OR` are used to combine multiple conditions.

- **MongoDB Queries**
  - MongoDB uses a different syntax but achieves the same results as SQL.
  - Instead of `SELECT`, MongoDB uses `find()` to retrieve documents.
  - Conditions are specified using JSON-like syntax, such as `{age: {$gt: 33}}` for ages greater than 33.
  - Regular expressions, like `/Joe/`, are used for pattern matching in MongoDB.
  - The `$or` operator is explicitly used to combine conditions, showing MongoDB's functional style.

- **Key Points**
  - MongoDB's syntax is more like programming with functions, which can be more intuitive for developers familiar with coding.
  - Both SQL and MongoDB allow for complex queries, but the way they are written differs significantly.
  - Understanding these differences is crucial for working effectively with both databases.

6 / 11: Query Operators

Query Operators

Command	Description
$regex	Match by any PCRE-compliant regular expression string (or just use the // delimiters as shown earlier)
$ne	Not equal to
$lt	Less than
$lte	Less than or equal to
$gt	Greater than
$gte	Greater than or equal to
$exists	Check for the existence of a field
$all	Match all elements in an array
$in	Match any elements in an array
$nin	Does not match any elements in an array
$elemMatch	Match all fields in an array of nested documents
$or	or
$nor	Not or
$size	Match array of given size
$mod	Modulus
$type	Match if field is a given datatype
$not	Negate the given operator check

6 / 11

- **$regex**: *Matches strings using regular expressions.* Useful for pattern matching within text fields, allowing for flexible searches.
- **$ne**: *Not equal to.* Filters documents where the field value is not equal to the specified value.

- **$lt**: *Less than.* Finds documents where the field value is less than the specified value.
- **$lte**: *Less than or equal to.* Similar to $lt but includes equality.
- **$gt**: *Greater than.* Retrieves documents with field values greater than the specified value.
- **$gte**: *Greater than or equal to.* Includes equality in the comparison.
- **$exists**: *Checks for the existence of a field.* Useful for determining if a field is present in documents.
- **$all**: *Matches all elements in an array.* Ensures all specified elements are present in the array field.
- **$in**: *Matches any elements in an array.* Finds documents where the field value matches any value in the specified array.
- **$nin**: *Does not match any elements in an array.* Opposite of $in, excludes documents with matching values.
- **$elemMatch**: *Matches all fields in an array of nested documents.* Useful for complex queries on arrays of documents.
- **$or**: *Logical OR.* Combines multiple conditions, returning documents that satisfy at least one.
- **$nor**: *Logical NOR.* Returns documents that do not satisfy any of the specified conditions.
- **$size**: *Matches arrays of a given size.* Filters documents based on the number of elements in an array.
- **$mod**: *Modulus.* Finds documents where the field value divided by a divisor has a specified remainder.
- **$type**: *Matches if a field is a given datatype.* Useful for ensuring data type consistency.
- **$not**: *Negates the given operator check.* Inverts the result of a query condition, useful for excluding specific matches.

7 / 11: Update Operations

Update Operations

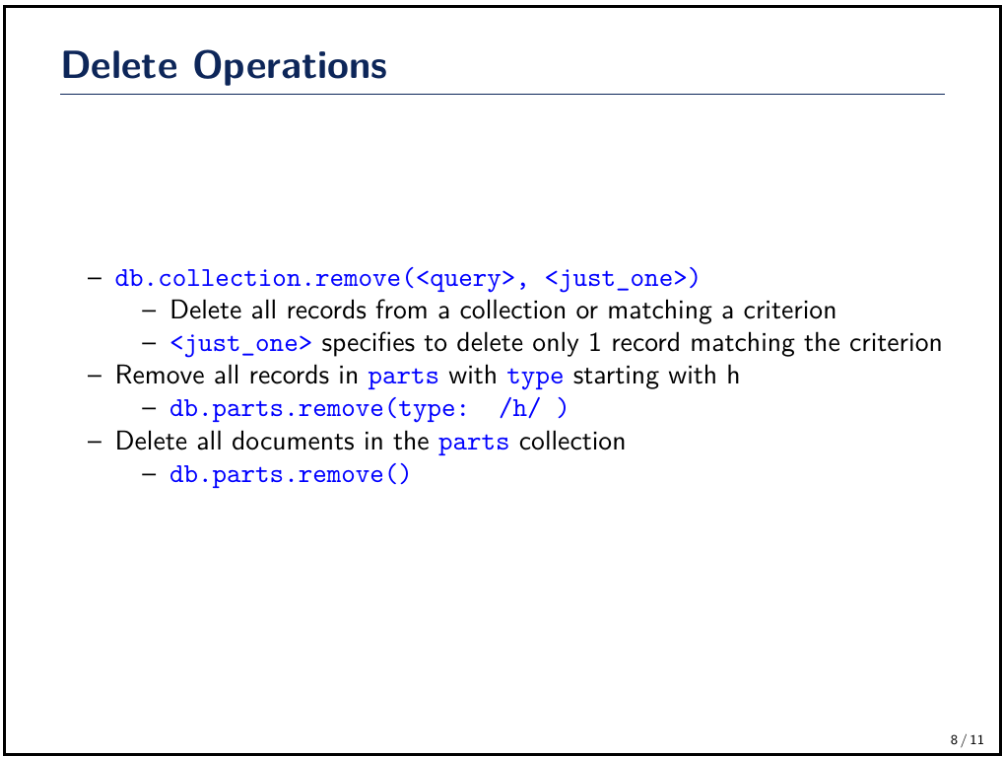
- db.collection.insert(<document>)
  - Omit the _id field to have MongoDB generate a unique key
    db.parts.insert({type: "screwdriver", quantity: 15})
    db.parts.insert({_id: 10, type: "hammer", quantity: 1})
- db.collection.save(<document>)
  - Updates an existing record or creates a new record
- db.collection.update(<query>, <update>, upsert: true)
  - Will update 1 or more records in a collection satisfying query
- db.collection.findAndModify(<query>, <sort>, <update>, <new>, <fields>, <upsert>)
  - Modify existing record(s)
  - Retrieve old or new version of the record

7 / 11

- **Update Operations**
  - **`db.collection.insert(<document>)`**
    - This command is used to add a new document to a MongoDB collection. If you don't specify an `_id` field, MongoDB will automatically generate a unique identifier for the document. This is useful because it ensures that each document can be uniquely identified within the collection.
    - *Example:* The command `db.parts.insert({type: "screwdriver", quantity: 15})` adds a new document with a type of "screwdriver" and a quantity of 15. If you specify an `_id`, like in `db.parts.insert({_id: 10, type: "hammer", quantity: 1})`, MongoDB will use that as the document's unique identifier.
- **`db.collection.save(<document>)`**
  - This operation is versatile because it can either update an existing document or create a new one if it doesn't already exist. It's a convenient way to ensure that a document is present in the collection, either by updating it or inserting it.
- **`db.collection.update(<query>, <update>, {upsert: true})`**
  - This command updates one or more documents that match a specified query. The `{upsert: true}` option is particularly important because it tells MongoDB to insert a new document if no existing documents match the query. This is useful for ensuring that data is present in the collection even if it wasn't there before.
- **`db.collection.findAndModify(<query>, <sort>, <update>, <new>, <fields>, <upsert>)`**
  - This is a powerful command that not only modifies existing documents but also allows you to retrieve either the old or new version of the document after the update. This can be useful for operations where you need to know the state of a document before

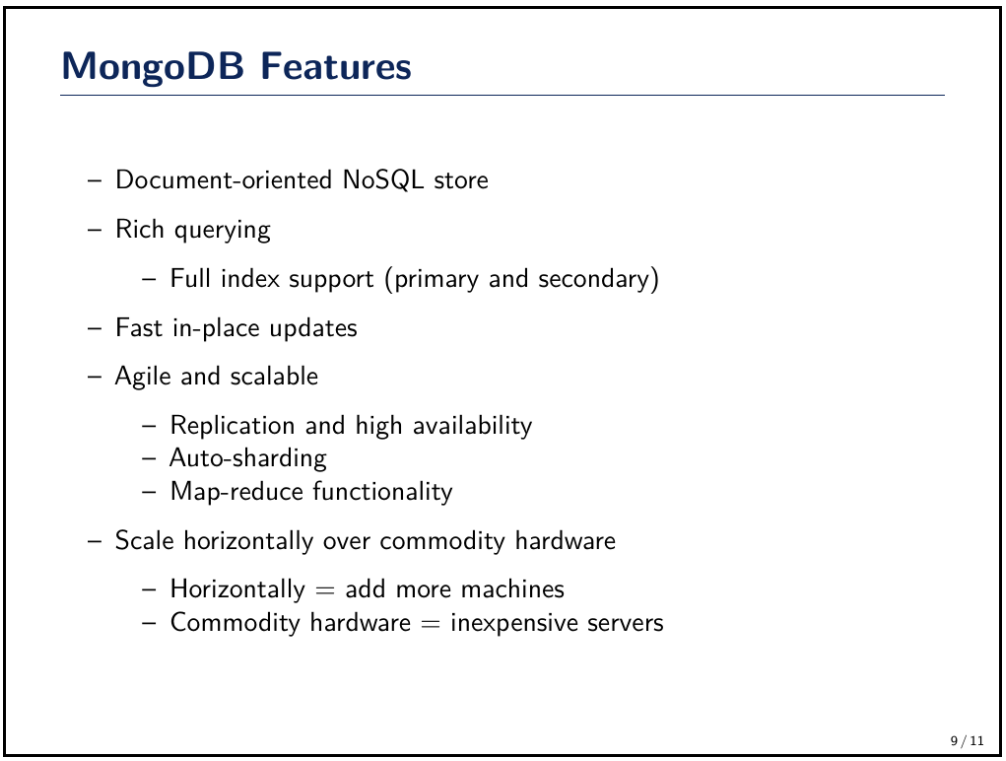
and after an update. The additional parameters like `<sort>`, `<fields>`, and `<upsert>` provide further control over how the operation is executed and what data is returned.

8 / 11: Delete Operations



- **Delete Operations**
  - The `db.collection.remove(<query>, <just_one>)` command is used to delete documents from a collection in a database.
    - The `<query>` parameter specifies the condition that documents must meet to be deleted. If you want to delete all documents that match a certain condition, you would specify that condition here.
    - The `<just_one>` parameter is a boolean that, when set to true, ensures that only the first document that matches the query is deleted. If it is not specified or set to false, all documents matching the query will be deleted.
- **Remove all records in `parts` with `type` starting with h**
  - The command `db.parts.remove(type: /h/ )` is used to delete documents from the `parts` collection where the `type` field starts with the letter "h".
  - The `/h/` is a regular expression that matches any string starting with "h". This is useful for pattern matching within strings.
- **Delete all documents in the `parts` collection**
  - The command `db.parts.remove()` is used to delete all documents in the `parts` collection.
  - This operation does not require a query parameter, as it is intended to remove every document in the collection, effectively clearing it out.

9 / 11: MongoDB Features

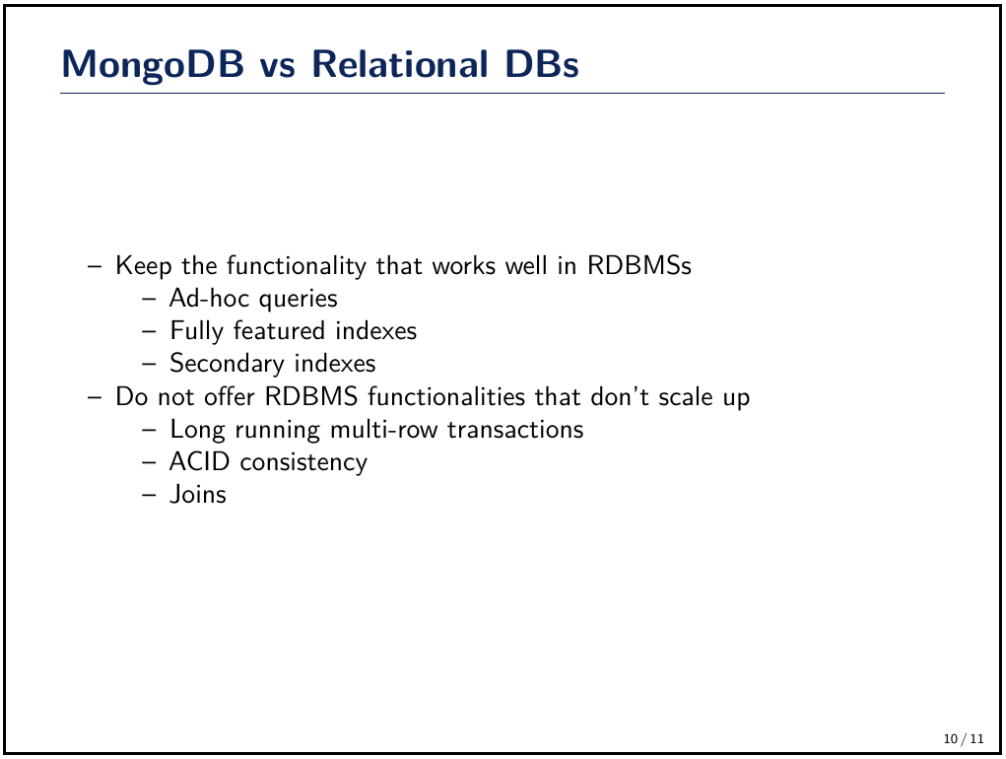


- **MongoDB Features**
- **Document-oriented NoSQL store**
  - MongoDB is a type of database that stores data in a flexible, JSON-like format called documents. This is different from traditional databases that use tables and rows. The document-oriented approach allows for more complex data structures and makes it easier to store and retrieve data that doesn't fit neatly into a table.
- **Rich querying**
  - *Full index support (primary and secondary)*: MongoDB supports indexing, which means you can create indexes on any field in a document to make queries faster. Primary indexes are automatically created on the unique identifier of each document, while secondary indexes can be created on other fields to optimize search operations.
- **Fast in-place updates**
  - MongoDB allows you to update parts of a document without having to rewrite the entire document. This makes updates faster and more efficient, especially when dealing with large datasets.
- **Agile and scalable**
  - *Replication and high availability*: MongoDB can replicate data across multiple servers, ensuring that your data is always available even if one server fails.
  - *Auto-sharding*: This feature automatically distributes data across multiple servers, which helps manage large datasets and improves performance.

  - *Map-reduce functionality*: MongoDB supports map-reduce, a programming model used for processing large data sets with a distributed algorithm on a cluster.

- **Scale horizontally over commodity hardware**
  - *Horizontally = add more machines*: Instead of upgrading a single server, you can add more servers to handle increased load, which is known as horizontal scaling.
  - *Commodity hardware = inexpensive servers*: MongoDB is designed to run on affordable, off-the-shelf servers, making it cost-effective to scale your database infrastructure.

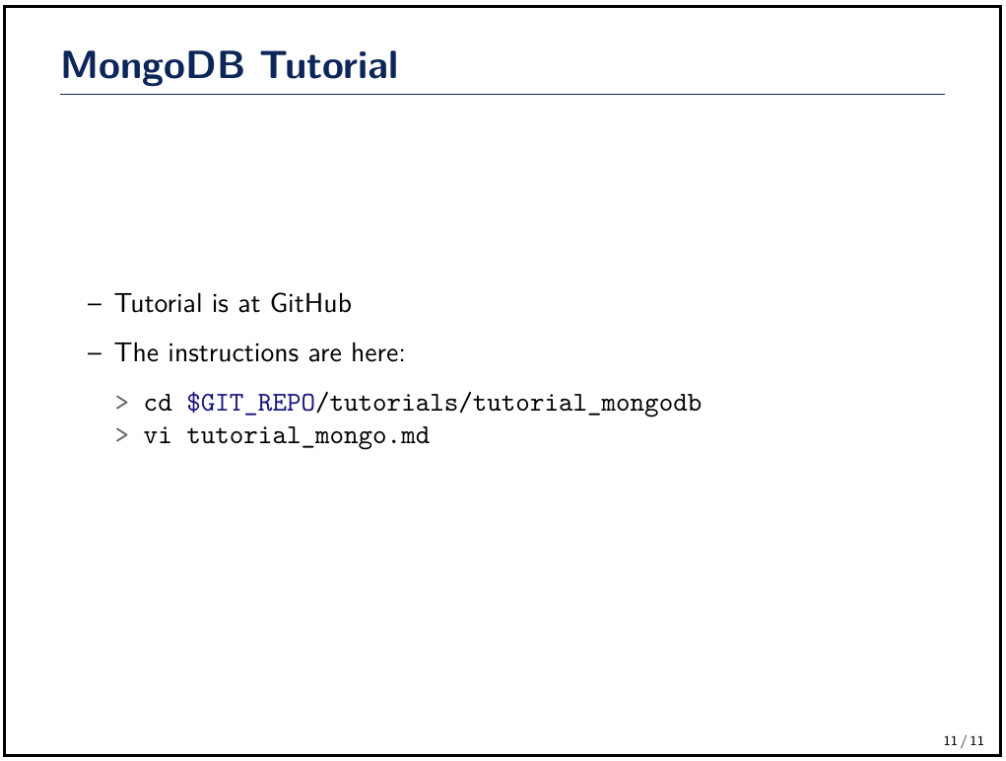
10 / 11: MongoDB vs Relational DBs



- **Keep the functionality that works well in RDBMSs**
  - **Ad-hoc queries**: MongoDB supports ad-hoc queries, which means you can search for data in a flexible way. This is similar to how you can use SQL in relational databases to find specific information without needing to predefine the queries. This flexibility is important for applications where the data retrieval needs can change over time.
  - **Fully featured indexes**: Indexes are crucial for speeding up data retrieval. MongoDB, like relational databases, allows you to create indexes on fields to make queries faster. This is especially useful when dealing with large datasets, as it helps in quickly locating the required data without scanning the entire database.
  - **Secondary indexes**: In addition to primary indexes, MongoDB supports secondary indexes, which allow you to index additional fields. This is beneficial for optimizing queries that involve multiple fields, providing more efficient data access patterns.
- **Do not offer RDBMS functionalities that don't scale up**
  - **Long running multi-row transactions**: Traditional relational databases support complex transactions that can involve multiple rows and tables. However, these can

become a bottleneck when scaling up, as they require locking mechanisms that can slow down performance. MongoDB opts for simpler transaction models that are more scalable.
  - **ACID consistency**: While ACID (Atomicity, Consistency, Isolation, Durability) properties ensure reliable transactions in relational databases, they can limit scalability. MongoDB uses a different approach, focusing on eventual consistency, which allows for better performance and scalability in distributed systems.
  - **Joins**: Joins are used in relational databases to combine data from different tables. However, they can be resource-intensive and slow down performance as the database grows. MongoDB avoids joins by using a document-based model, where related data is often stored together, reducing the need for complex join operations.

11 / 11: MongoDB Tutorial



- **MongoDB Tutorial**
  - This slide introduces a tutorial on MongoDB, which is a popular NoSQL database. MongoDB is known for its flexibility and scalability, making it a great choice for handling large volumes of data. This tutorial will likely cover the basics of using MongoDB, including how to set up a database, perform CRUD operations (Create, Read, Update, Delete), and possibly more advanced topics like indexing and aggregation.
- **Tutorial is at GitHub**
  - The tutorial is hosted on GitHub, a platform widely used for version control and collaboration. This means you can access the tutorial files, contribute to them, or even fork the repository to make your own modifications. GitHub is a valuable resource for developers to share and collaborate on projects.
- **The instructions are here:**

- The slide provides a couple of command-line instructions to access the tutorial files.
- *`cd $GIT_REPO/tutorials/tutorial_mongodb`*: This command changes the directory to where the MongoDB tutorial is located within your cloned GitHub repository. `$GIT_REPO` is a placeholder for the path to your local copy of the repository.
- *`vi tutorial_mongo.md`*: This command opens the `tutorial_mongo.md` file using `vi`, a text editor available on Unix-like systems. This file likely contains the step-by-step instructions or content of the MongoDB tutorial. If you're not familiar with `vi`, you might want to use another text editor you're comfortable with.